The Complete Technical Guide

Learning Muscle Forces with Deep Reinforcement Learning

Every decision, every parameter, every result — explained from first principles

Mohamed Naceur Hammami

Tunisia Polytechnic School (EPT)

August 18, 2026

How to read this document. It is written to be answerable at every level. Ordinary text carries the technical account. Boxes marked **IN PLAIN LANGUAGE** restate the same idea with no assumed background — if a section is opaque, read its box first and then the text. Boxes marked **WHY IT IS THIS WAY** explain a design decision rather than describing it. Boxes marked **WHAT GOES WRONG** record a failure that actually occurred in this project and how it presented.
Nothing here is aspirational. Every number is measured, every parameter value is the one in the code, and every failure described actually happened.

Contents

1 Why This Work Exists

1.1 Start with the body

Raise your arm and touch something in front of you. That felt like a single decision. It was not. Somewhere between deciding and moving, your nervous system settled dozens of separate questions: how hard should the biceps pull, how hard the triceps, the three heads of the deltoid, and so on, renegotiated continuously for the whole half-second the movement takes.

Knowing those individual muscle forces is genuinely useful:

- **Prosthetics and exoskeletons.** A powered device that assists a weakened arm must know what the arm is trying to do, and how hard, in order to supply the difference.
- **Rehabilitation.** After a stroke, recovery is often tracked by how movement is *produced*, not only whether the hand arrives. Two patients can touch the same target with completely different muscle strategies, and the difference is clinically meaningful.
- **Surgery and injury.** Whether a tendon transfer will work, or whether a joint is being overloaded, depends on force in individual muscles — not on the visible motion.

The difficulty is that muscle force is almost never measured. Putting a sensor on a tendon inside a living person is not routinely possible. So the forces are *computed* from the movement, and that computation is the subject of this work.

1.2 The counting problem that makes it hard

Here is the whole difficulty in one observation.

Our model of the arm has **four ways to move** — the shoulder can swing forward, out to the side, and rotate; the elbow can bend — and **nine muscles** to move them with. Nine controls, four things to control.

IN PLAIN LANGUAGE. Imagine nine people holding ropes attached to a single heavy door, and you want the door at one particular angle.
They could all pull gently together. Or three could pull hard while six rest. Or — wastefully — four could pull one way while five pull the other, mostly cancelling, everyone exhausted, the door in exactly the same place.
Stand across the room and watch only the door. All three look identical. The door angle does not tell you who was pulling.

That is the **muscle redundancy problem**, and it is not a limitation of our instruments. It is a mathematical fact: with more muscles than degrees of freedom, infinitely many force combinations produce exactly the same movement. Watching the movement can never, even in principle, tell you the forces.

WHY IT IS THIS WAY. This is why the thesis cannot be answered by simply measuring how well a controller reaches its target. Reaching accuracy is the *door angle*. Two controllers with identical reaching accuracy can be using completely different muscles. Everything in this project that looks like a detour — the force comparison, the co-contraction analysis, the synergy decomposition — exists because the obvious measurement cannot answer the question.

1.3 The classical answer, and what it costs

Since the movement does not determine the forces, an extra rule is needed to pick one answer from the infinitely many. The rule used in biomechanics since Crowninshield and Brand (1981) is **minimum effort**: among all combinations that produce the required movement, choose the one where the muscles work least hard.

Written out, the method — called **static optimisation** — has two steps.

Step one, inverse dynamics. From the recorded motion and the limb's mass properties, compute the twisting force (torque) needed at each joint. This step is exact. There is one answer.

Step two, load sharing. Distribute those joint torques among the muscles according to the minimum-effort rule:

$$\min_{f} \sum_{i=1}^{9} \left(\frac{f_i}{F_{\max,i}} \right)^2 \quad \text{subject to} \quad R^\top f = \tau, \qquad 0 \le f_i \le F_{\max,i}.$$

IN PLAIN LANGUAGE. Reading that formula in words:
Choose the nine muscle forces f that make the total effort as small as possible — where each muscle's effort is its force as a fraction of its own maximum strength, squared.
Subject to two conditions. First, the forces must actually produce the required joint torques τ (that is $R^\top f = \tau$, where R holds each muscle's leverage about each joint). Second, no muscle may pull harder than it can, and none may push — muscles only pull. That is $0 \le f_i \le F_{\max,i}$.
Why square the effort? Squaring punishes large values disproportionately, so sharing work among several muscles beats making one muscle do everything. It also makes two people pulling against each other very expensive, because both contributions count and neither cancels in the cost.

This method works and is standard. Its drawback is that it solves a fresh optimisation problem *at every instant of the movement.* That is fine for analysing a recording afterwards. It is awkward inside a device that must respond in milliseconds.

1.4 Why anyone would try machine learning here

A neural network has the opposite cost profile: expensive to train once, then very cheap to use — a fixed number of multiplications, identical for every input, with no iteration and no convergence check.

IN PLAIN LANGUAGE. Classical optimisation is like solving a puzzle from scratch every time you are asked. A trained network is like having memorised the *pattern* of answers: it responds instantly, but only as well as its training taught it.
The obvious question is whether the memorised pattern is actually right — and that is exactly the question this thesis asks.

WHAT GOES WRONG. The original motivation was speed, and speed turned out not to survive measurement. The classical solve was timed at 2165 µs against the network's 295 µs, a 7.3× advantage — but that timed a *different* calculation (the muscle-model inversion inside the simulator, not the load-sharing problem), and both sides were unoptimised Python. Re-timed properly, a direct solve of the actual load-sharing problem takes 24 µs, about **ten times faster** than the network. The speed argument was withdrawn. Chapter 6 gives the numbers.

1.5 What this thesis actually asks

The question. For a reaching movement of a nine-muscle arm, does a controller trained by deep reinforcement learning produce the *same per-muscle forces* as classical static optimisation?

And, because the answer turned out to have two halves that behave differently:

1. Does it recruit the *same muscles in the same proportions*? — the *pattern* of the solution.
2. Does it use the *same amount of force*? — the *economy* of the solution.

The short answer, established in Chapter 6: the pattern agrees well; the economy did not, until the objective was changed to ask for it, at which point it largely did too.

1.6 An important boundary on what can be claimed

One limitation shapes everything and should be understood before any result is read.

Both the learned controller and the classical reference are computed inside the same physics simulator, against the same model of muscle. Where they agree, two computations over the same approximation agree. **No real arm, no electromyography, and no human subject enters this work at any point.**

Furthermore, static optimisation is itself a *theory* of how the nervous system shares load — a well-established and useful one, but contested. A controller that disagrees with it is not thereby shown to be unphysiological.

WHY IT IS THIS WAY. This is stated first rather than buried in a limitations section because it sets the meaning of every subsequent number. "Agreement of $r = 0.94$ with the classical solution" means *this controller resembles the standard computational method.* It does not mean *this controller resembles a human.* Those are different claims and only the first is supported.

2 How Deep Reinforcement Learning Works

This chapter assumes nothing. If you already know actor–critic methods, skip to §2.6, where SAC is described as this project uses it.

2.1 Learning by consequence

Most machine learning is *supervised*: you are shown thousands of examples with the right answer attached, and you learn to reproduce them. That is unavailable here — nobody can supply the correct muscle forces, because that is the very thing we cannot measure.

Reinforcement learning removes the need for correct answers. Instead:

1. The learner observes the current situation.
2. It chooses an action.
3. The world changes, and it receives a number — the *reward* — saying how good that was.
4. Repeat, millions of times, adjusting so that high-reward behaviour becomes more likely.

IN PLAIN LANGUAGE. It is learning the way you learn to ride a bicycle: nobody tells you the correct handlebar angle. You try, you wobble, you notice which corrections stop the wobbling, and you do more of those. The wobbling is the reward signal.

2.2 The vocabulary, applied to this arm

Term	General meaning	In this project
Agent	the thing being trained	a neural network, 393,750 numbers
Environment	everything it acts on	the simulated arm and a target point
State	the true situation	joint angles, speeds, muscle states
Observation	what the agent is told	38 numbers (§4.3.1)
Action	what it outputs	9 numbers in $[0, 1]$, one per muscle
Reward	the score after each action	a formula, seven terms (§4.3.2)
Episode	one attempt	one 2-second reach, 100 decisions
Policy	the learned rule	observation → action; this *is* the trained controller
Return	total reward over an episode	what the agent actually maximises
Value	expected return from a state	the critic's estimate (§2.3)

WHY IT IS THIS WAY. Note the distinction between *state* and *observation*. The state is what is true; the observation is what the agent is permitted to see. Here they differ deliberately: the agent sees muscle lengths, forces and activations, joint angles and speeds, and the direction to the target — but not, for instance, the randomised body masses used during training. It must cope without knowing them, which is what makes the controller robust rather than brittle.

2.2.1 The reward is a scoring rule, not an instruction

This distinction causes more trouble than any other idea in the subject, so it is worth stating sharply.

The agent is **never told how to move**. It is told only how good the outcome was. Any behaviour that scores highly will therefore be found, including behaviour the designer never imagined and would not endorse.

WHAT GOES WRONG. In this project the reward was, for a period, maximised by **deliberately destroying the simulated arm**. Every per-step reward was zero or negative, and a numerical blow-up ended the episode with no penalty. In this framework an episode that *ends* accrues nothing further — its remaining value is exactly zero — while surviving the full 100 steps accumulated about −20. Quitting was therefore worth more than participating.
The agent found this. Measured over 50 evaluation episodes: **100 % ended in a deliberate blow-up at step 7 of 100.** Not one ran to completion.
The algorithm was not broken. It did exactly what it was asked. The request was wrong. Full account in §6.2.

2.3 Why a "value" is needed, and what the discount factor does

An action's consequences are not immediate. Pulling a muscle now changes where the hand is half a second later. So the agent cannot judge an action by the reward that immediately follows it — it must judge by the reward that *eventually* follows.

That total is the **return**:

$$G_t \;=\; r_t \;+\; \gamma\, r_{t+1} \;+\; \gamma^2 r_{t+2} \;+\; \gamma^3 r_{t+3} + \cdots$$

The number γ (gamma), between 0 and 1, is the **discount factor**. It shrinks rewards that arrive later.

IN PLAIN LANGUAGE. γ answers: "how far into the future should I care?"
At $\gamma = 0$ the agent is purely greedy — only the next reward matters. At γ close to 1 it weighs events far ahead almost as heavily as now.
A useful rule of thumb: the agent effectively plans about $1/(1-\gamma)$ steps ahead. At $\gamma = 0.99$ that is 100 steps. At $\gamma = 0.957$ it is 23.

WHY IT IS THIS WAY. Remember that rule of thumb. **The discount factor turned out to be the single most consequential setting in this entire project** — worth more than the choice of algorithm — and the reason is exactly this horizon arithmetic. The default value made the agent plan over 100 steps when the reach it was learning completes in about 25. Chapter 6, §6.3.

Because the future is unknown, the agent learns to *estimate* the return. That estimator is the **critic** (or value function): given a situation and an action, it predicts the total reward that will follow. The critic is what makes learning possible from delayed consequences — and, as we will see, a critic fed noisy targets is what makes learning fail.

2.4 Actor and critic

Modern methods train two networks together:

- The **actor** (the policy) chooses actions. This is the thing we ultimately keep and deploy.

- The **critic** estimates how good those choices are.

They bootstrap each other: the critic learns to predict returns from experience, and the actor adjusts to prefer actions the critic rates highly.

IN PLAIN LANGUAGE. A coach and an athlete. The athlete performs; the coach judges. The athlete changes technique to earn better judgements; the coach refines their judgement by watching what actually happens. Both improve together — and if the coach is unreliable, the athlete trains toward the wrong thing. That failure mode is not hypothetical here; it is what the discount factor caused.

2.5 Off-policy learning and the replay buffer

An **off-policy** method stores every experience in a large memory — the *replay buffer* — and re-learns from it repeatedly. An **on-policy** method uses each batch of experience once and discards it.

IN PLAIN LANGUAGE. Off-policy keeps a notebook of every attempt ever made and re-reads it constantly. On-policy makes a batch of attempts, learns from them, throws the notebook away, and starts fresh.
Re-reading is far more efficient when attempts are expensive, which is why three of our four algorithms do it.

The **replay ratio** — gradient updates performed per environment step — matters enormously and is easy to get wrong.

WHAT GOES WRONG. This project ran with four environments in parallel and one gradient update per rollout iteration. Because one iteration over four environments collects *four* transitions, the actual ratio was 0.233 updates per step rather than the standard ≈ 1. Every experiment was doing roughly a quarter of its intended training, invisibly. Measured directly in §5.3.1.

2.6 The four algorithms

2.6.1 DDPG — Deep Deterministic Policy Gradient

The oldest of the four. One actor, one critic. The actor outputs a single definite action; exploration is added by injecting random noise into it.

Its known weakness is *overestimation*: the critic systematically rates actions better than they are, the actor chases those inflated ratings, and training becomes unstable.

2.6.2 TD3 — Twin Delayed DDPG

DDPG with three corrections. It trains **two** critics and always believes the more pessimistic one, which counters overestimation. It updates the actor less often than the critics ("delayed"), and it smooths the critic's target by adding small noise, so the actor cannot exploit a sharp spike in the critic's estimate.

2.6.3 SAC — Soft Actor-Critic

Also uses twin critics, but differs in one important way: its policy is **stochastic**. It outputs a probability distribution over actions rather than a single action, and it is explicitly rewarded for

keeping that distribution spread out. The quantity being maximised is

$$\sum_t \Big(r_t \; + \; \alpha \, \mathcal{H}[\pi(\cdot \mid s_t)] \Big),$$

where $\mathcal{H}$ is entropy — a measure of randomness — and α its weight.

IN PLAIN LANGUAGE. Entropy is a measure of how undecided something is. A policy that always does exactly the same thing has zero entropy. One that picks randomly has high entropy. SAC pays the agent a bonus for staying undecided. The purpose is to stop it committing early to a mediocre habit before exploring alternatives.
The weight α is tuned automatically to hit a chosen **target entropy**. The library default sets that target to minus the number of controls — here -9, because there are nine muscles.

WHY IT IS THIS WAY. That default is worth pausing on. It scales with the *number of actions*, not with anything about the task. A nine-muscle arm gets more forced randomness than a two-joint one, regardless of whether the task needs precision.
For a long time this project believed that default was the cause of its inability to hold position — the mechanism fits perfectly, and every one of the best hyperparameter trials agreed on a much lower value. **A direct ablation later proved that belief wrong.** The story is told in §6.3.1, because being wrong for a good reason is instructive.

2.6.4 PPO — Proximal Policy Optimisation

Structurally different: on-policy. It collects a batch of experience, improves the policy while limiting how far it may change in one step (the "proximal" constraint, which prevents destructive updates), then discards the batch.

Simple and robust, but much less sample-efficient — it needs far more interaction for the same skill, which matters when each interaction requires simulating physics.

2.6.5 Summary of the differences that matter here

	Critics	Policy	Reuses data	Exploration by
DDPG	1	deterministic	yes	added noise
TD3	2 (min)	deterministic	yes	added noise
SAC	2 (min)	stochastic	yes	entropy bonus
PPO	1 (value)	stochastic	no	policy's own randomness

WHY IT IS THIS WAY. Only SAC has a *target entropy*, because only SAC has an entropy bonus to tune. This asymmetry matters for the algorithm comparison: there is no single setting one can hold equal across all four, which is part of why fair comparison between reinforcement learning algorithms is harder than it looks (§6.5).

3 The Physical Model

3.1 What is being simulated

A right arm, hanging from a fixed shoulder, in three dimensions, under gravity. It has three rigid bodies (thorax anchor, upper arm, forearm), four rotational degrees of freedom, and nine muscles. Physics is computed by **MuJoCo**, a simulator widely used in robotics and biomechanics.

3.1.1 The four degrees of freedom

Joint axis	Movement	Actuated	Note
Shoulder flexion	arm forward/backward	yes	
Shoulder abduction	arm out to the side	yes	
Shoulder rotation	upper arm twisting about itself	**no**	constrained, see below
Elbow flexion	forearm bending	yes	

3.1.2 Why shoulder rotation is restrained

This is a substantive modelling decision, made on measurement rather than convenience.

A static test asked, across many arm postures, whether the nine muscles could hold the arm still against gravity. Shoulder flexion, abduction and elbow flexion were held essentially perfectly — residual torque 0.00 N m to 0.02 N m, using only about 8 % of available strength. Shoulder *rotation* could not be held: 0.34 N m residual, and only 59 % of postures holdable at all.

The cause is anatomical. The muscles that rotate the upper arm about its own axis are the rotator cuff — supraspinatus, infraspinatus, subscapularis, teres minor — and none is in the nine-muscle set.

IN PLAIN LANGUAGE. The model had a joint that no muscle could control. The arm sagged around that axis by up to 0.19 m at the hand no matter what the controller did.
That is not something a learning algorithm can fix. It is a missing part. Leaving it in would have meant every experiment carried a large error that had nothing to do with learning.

The fix constrains that axis near neutral (range ±0.05 rad, with stiffness and damping) and samples targets accordingly. Afterwards 94–100 % of postures were holdable with zero residual. The degree of freedom was *retained* rather than deleted, so the observation vector stays 38-dimensional and the previously measured policy-size and latency figures remain valid.

3.1.3 The nine muscles

Muscle	Action	Group
biceps long	bends elbow, assists shoulder	elbow flexor
biceps short	bends elbow	elbow flexor
brachialis	bends elbow	elbow flexor
triceps long	straightens elbow, assists shoulder	elbow extensor
triceps lateral	straightens elbow	elbow extensor
triceps medial	straightens elbow	elbow extensor
deltoid anterior	lifts arm forward	shoulder
deltoid medial	lifts arm sideways	shoulder
deltoid posterior	pulls arm backward	shoulder

Anatomical parameters — optimal fibre length, tendon slack length, peak isometric force, pennation angle — follow Holzbaur et al. (2005).

3.2 How a muscle is modelled: the Hill model

A muscle is not a motor. The force it produces depends on three things at once: how strongly it is being driven, how stretched it currently is, and how fast it is changing length.

The standard description represents a muscle as three mechanical elements:

- **Contractile element (CE)** — the active part, which pulls when stimulated.
- **Parallel elastic element (PEE)** — a spring alongside it, representing the passive stiffness of the tissue, which resists stretching even when the muscle is completely relaxed.
- **Series elastic element (SEE)** — the tendon, a spring in line with the muscle connecting it to bone.

Two relationships govern the contractile element.

Force–length. A muscle produces its greatest force at one particular length, and less at any other, whether shorter or longer.

Force–velocity. A muscle shortening quickly produces less force than one held still. A muscle being stretched while active can resist more than its isometric maximum.

IN PLAIN LANGUAGE. Both are familiar from your own body. Force–length is why a push-up is hardest at the very bottom, where the chest muscles are at an awkward length. Force–velocity is why you can lower a weight slowly that you could never lift: muscles are stronger resisting than contracting.
The practical consequence is that "how much force can this muscle make?" has no single answer. It depends on posture and on how fast you are moving.

Total force is

$$F \;=\; \left[\, a \cdot f_L(\tilde{l}) \cdot f_V(\tilde{v}) \;+\; f_{PE}(\tilde{l}) \,\right] \cdot F_{\max} \cos\alpha,$$

where $a \in [0,1]$ is activation, $\tilde{l}$ and $\tilde{v}$ are normalised length and velocity, $F_{\max}$ is peak isometric force, and α is the pennation angle.

WHAT GOES WRONG. Measured in this model, the parallel elastic element never engages. Probing every muscle at zero activation across sampled trajectories returns a passive force of exactly zero in all states. The reason is visible in the operating lengths: no muscle exceeds 0.67 of its declared length range, because the ranges were measured and then padded, and the passive term only rises near the top of that range.
So "complete Hill model" accurately describes what the model file specifies but overstates what is active: the contractile and series-elastic elements do the work; the parallel element is inert. The force–length curve is also exercised only on its ascending limb and plateau, never its descending limb.
One favourable consequence: the load-sharing analysis bounds muscle force below by zero, which would be *wrong* if a passive floor existed — a muscle cannot produce less than its passive tension. Because that tension is zero throughout, the bound is correct and the reported effort ratios are not inflated by it.

3.3 Activation dynamics: the difference between `ctrl` and `act`

This distinction is small, easy to miss, and caused a real bug.

The agent outputs a **command**. The muscle has an internal **activation state** that follows that command with a lag — real muscle cannot switch on instantly, and the model reproduces this with first-order dynamics.

In MuJoCo these are separate variables:

Variable	Meaning	Note
`data.ctrl`	the command sent in	what the policy outputs
`data.act`	the activation state	lags `ctrl`; *this* drives force
`data.actuator_force`	resulting force	computed from `act`, length, velocity

Force is $\text{gain}(l, v) \cdot \texttt{act} + \text{bias}(l, v)$ — a function of `act`, not of `ctrl`.

WHAT GOES WRONG. When building the conventional controller, the achievable force range of each muscle was probed by setting the command to 0 and then to 1 and reading the resulting force. This produced *no change at all*, because recomputing the model does not integrate `ctrl` into `act` — the activation state was untouched, so the force was identical both times.
The measured achievable range was therefore zero-width, and the controller emitted zero activation and never moved. Probing `act` instead of `ctrl` fixed it immediately. §4.7.

3.4 Moment arms: how muscle force becomes joint torque

A muscle pulls along its own line. What that does to a joint depends on its **moment arm** — the leverage it has about that axis, exactly like the length of a spanner.

Collecting these gives the **moment-arm matrix** R, with one row per muscle and one column per degree of freedom. The joint torque produced by a set of muscle forces is

$$\tau \;=\; R^{\top} f.$$

IN PLAIN LANGUAGE. R is the translation table between "how hard each muscle pulls" (nine numbers) and "how hard each joint is twisted" (four numbers).
Nine into four: that is the redundancy problem in one line. Many different nine-number inputs give the same four-number output.

WHY IT IS THIS WAY. This equation is the pivot of the entire thesis. The classical method solves it *backwards* — given the torque τ, find forces f — and needs the minimum-effort rule because the answer is not unique. The learned controller never solves it at all; it outputs f directly and the physics produces whatever τ follows.

The comparison in Chapter 6 works by taking the τ the policy actually produced, handing it to the classical method, and asking what *it* would have done. Both methods then face an identical problem, and any difference is purely a difference of strategy.

One technical detail matters: MuJoCo stores R in a compressed (sparse) format from version 3.2 onward. Code that assumes the older dense layout reads garbage. This broke the conventional controller until it was densified explicitly.

4 How the Code Works

4.1 The map

Twenty-odd Python files, but only six carry the science. Everything else is orchestration or diagnosis.

File	Responsibility
Core — the science	
`arm.xml`	the physical model: bodies, joints, muscles, geometry
`config.py`	every constant in one place; single source of truth
`rl/environment.py`	the task: observation, reward, episode rules
`rl/train.py`	the training loop, checkpointing, learning curve
`rl/evaluate.py`	rollout and metrics — produces every reported number
`force_comparison.py`	**the central analysis**: DRL vs classical
Supporting analyses	
`run_conventional.py`	a classical controller solving the same task
`run_synergies.py`	muscle-synergy decomposition
`analysis/synergies.py`	the NMF machinery
`bench_loadsharing.py`	like-for-like latency measurement
Verification — written to catch specific failures	
`verify_replay_ratio.py`	measures gradient updates per environment step
`verify_divergence.py`	audits simulator warnings against reported blow-ups
`test_reward_v3.py`	14 checks on the reward's properties
`oracle_feasibility.py`	can the muscles hold the arm at all?
Orchestration	
`run_parallel.py`	runs jobs across CPU and GPU concurrently
`reevaluate.py`	re-runs all evaluations after the evaluation-code fixes

4.2 How the parts talk to each other

```
config.py   (constants:  weights, budgets, seeds, paths)
   ↓ read by everything

arm.xml → environment.py → [obs 38] → algorithms.py → SB3 model
            ↑ [action 9]                              ↓
            +-------- train.py (loop, checkpoints) ---+

train.py → runs/<name>/{model.zip, vecnormalize.pkl, train_meta.json}
                    ↓
evaluate.py → metrics    force_comparison.py → effort ratio, per-muscle
                    ↓                                  ↓
                runs/*.json → build_figures*.py → the report
```

WHY IT IS THIS WAY. Two design rules hold throughout, and both were adopted after being violated.
One source of truth. Every constant lives in `config.py`. Nothing is written twice. When the effort weight changed, it changed in one place and every downstream script saw it.
Numbers reach the report only through files. No figure or table is typed by hand. Scripts write JSON; `build_figures*.py` reads that JSON. This is a direct response to discovering that the previous version of this thesis contained numbers generated by `numpy.random.normal` and never replaced.

4.3 `rl/environment.py` — the task

This is where the problem is actually defined. Everything else is machinery.

4.3.1 The observation: what the agent sees

38 numbers, assembled fresh each step:

Indices	Count	Content	Why the agent needs it
0–3	4	joint angles q (rad)	where the arm is
4–7	4	joint velocities $\dot{q}$ (rad/s)	whether it is moving, and how fast — required to decelerate
8–16	9	muscle lengths	determines available force via force–length
17–25	9	muscle forces (N)	what each muscle is currently doing
26–34	9	muscle activations	the internal state lagging the command
35–37	3	target minus hand position	where to go, as a vector

WHY IT IS THIS WAY. Two choices here are deliberate.
The target is given as a *difference* (target minus hand), not as an absolute coordinate. The agent therefore learns "move in this direction" rather than "go to this place", which generalises across the workspace instead of memorising locations.
Joint velocities are included because without them the task is not solvable. A controller that cannot perceive that it is moving cannot decide to stop — and stopping on target is precisely what this task requires.

4.3.2 The reward

$$r = \underbrace{w_4}_{\text{survival}} - \underbrace{w_1(\|e\|^2 + 0.5\|e\|)}_{\text{distance cost}} - \underbrace{w_2\,\overline{(f_i/F_{\max,i})^2}}_{\text{effort}} - \underbrace{w_3\,\overline{(\Delta a_i)^2}}_{\text{smoothness}} + \underbrace{w_7 e^{-\|e\|/s}}_{\text{precision bonus}} + \underbrace{w_5\,[\text{on target}]}_{\text{success}} - \underbrace{w_6\,[\text{blow-up}]}_{\text{penalty}}$$

Term by term:

Term	Weight	What it does and why it exists
Survival	$w_4 = 4.5$	A constant paid every step. Makes the per-step reward strictly positive, so ending an episode early forfeits value. Added specifically to kill the self-destruction exploit. A constant cannot reorder trajectories of equal length, so it does not disturb the other terms' tuning.
Distance	$w_1 = 1.0$	Quadratic *and* linear in error. Quadratic alone has no gradient near the target ($e^2 = 0.01$ at 10 cm), so policies hovered; the linear term keeps a pull at all ranges.
Effort	w_2	Mean squared force as a fraction of each muscle's maximum. This is the classical criterion in form. **Its weight is the subject of a whole experiment** (§6.7).
Smoothness	$w_3 = 0.5$	Penalises abrupt changes in command, discouraging jitter. Mean, not sum, so it is bounded by 1 like the effort term.
Precision bonus	$w_7 = 3.0$	$e^{-\|e\|/s}$ with $s = 0.15$ m. Negligible far away, steep close in — a dense, shaped version of the sparse success bonus, added because the sparse one was never once collected.
Success	$w_5 = 1.0$	Paid *every step* the hand is on target and settled, not once on arrival. This makes the task reach-*and*-hold.
Blow-up	$w_6 = 10.0$	One-off penalty for numerical divergence.

WHAT GOES WRONG. The precision bonus was originally $s = 0.05$ m. At the 0.23 m where the policy actually sat, $e^{-0.23/0.05} = 0.010$ — the bonus contributed 0.03 of a reward of about 4, i.e. nothing. It only started paying once the hand was already within 15 cm, a region the policy could not reach. It was a prize behind a wall. Setting $s = 0.15$ matched the scale to where the policy actually lived.

4.3.3 Episode rules

- **Truncation** at 100 agent steps (2 seconds).
- **Termination** only on numerical divergence.
- **Success does not end the episode** — arriving early means collecting the bonus for staying, which is the point.
- **Targets** are generated by picking random joint angles and computing where the hand would be. Every target is therefore reachable by construction — an earlier version sampled from a box, much of which lay outside the workspace.

4.3.4 Action repeat (frame skip)

The policy acts at 50 Hz while physics runs at 500 Hz: each command is held for 10 physics steps.

WHY IT IS THIS WAY. This looks like a detail and is worth an experiment's difference. At one action per physics step, a 2-second episode is 1000 decisions, so a million-step budget buys only 1000 reaching *attempts* — and the policy regressed toward the middle of the workspace from sheer lack of experience.
At 50 Hz the same budget buys about 10,000 attempts. And it is nearly free: a physics step costs 6.8 µs while a gradient update costs about 14 ms, so wall-clock is set by learning, not simulation. Ten times the physics adds roughly a minute to a 75-minute run.

4.4 `rl/train.py` — the training loop

Wraps Stable-Baselines3 with three additions.

Learning curve. Every 25,000 or 50,000 steps, run five deterministic episodes and record error, success, blow-up rate and episode length. It is a monitoring signal only; no reported number comes from it, because five episodes is far too noisy.

Checkpointing. Every 50,000 steps, save model weights, normalisation statistics, the replay buffer and progress. Written to a temporary file and atomically renamed, so an interrupted save cannot corrupt the previous one.

WHAT GOES WRONG. This exists because a forced operating-system restart destroyed a 75-minute run five minutes from completion. The original code saved only after training finished, so nothing was recoverable. Atomic renaming matters because a crash *during* a save would otherwise leave an unloadable half-written file where a good checkpoint used to be.

Best-model tracking. Saves whichever checkpoint scored best on a validation stream seeded differently from the reporting stream, so the reported number is not the one selection was performed on. (In practice all reported results use the *final* model, so this introduces no selection bias.)

4.5 `rl/evaluate.py` — where the numbers come from

Loads a trained policy, runs 50 deterministic episodes with domain randomisation off, and computes every metric.

Metric	Definition and purpose
final error	hand-to-target distance at the end. The headline accuracy number.
closest approach	smallest distance at any instant. Separates "could it get there" from "could it stop there".
drift	final minus closest. How far it wandered back out.
reach 5/10 cm	fraction of episodes *ending* within that distance
touch 2/5 cm	fraction reaching within that distance at any moment
energy	$\sum_i F_i^2$, a metabolic proxy
blow-up rate	fraction of episodes ending in divergence
episode length	mean steps survived; **exists to catch an agent that quits**
success rate	within 2 cm *and* nearly stationary at the end

WHY IT IS THIS WAY. The last two exist because of the self-destruction exploit. When it was occurring, the recorded error was still a plausible, slowly improving number, and nothing logged how *long* episodes lasted — so an agent quitting after 7 of 100 steps looked identical to one trying hard and plateauing. Episode length and blow-up rate make that failure impossible to hide.
Success rate is deliberately not a headline metric. The criterion is one that a controller *with privileged information* satisfies only about 7 % of the time. A measure that a near-best-possible controller fails 93 % of the time cannot discriminate between learned policies. Across five identically configured runs it read 0 %, 4 %, 6 %, 8 % and 20 % — a twentyfold spread that identifies it as noise.

WHAT GOES WRONG. Four defects were later found in this file, none producing arithmetically wrong numbers but two changing *which episodes* the numbers describe: the evaluation seed was dead code; the environment was reset twice per episode, so evaluation scored on every *other* target; the device setting was ignored; and a diverged episode could be scored a success. All are documented in §5.5 with what they changed.

4.6 `force_comparison.py` — the central analysis

This file answers the thesis question. Everything else supports it.

At every timestep of a policy rollout it solves the classical problem on the movement the policy just produced:

$$\min_{f} \sum_{i} \left(\frac{f_i}{F_{\max,i}} \right)^2 \quad \text{s.t.} \quad R^\top f = \tau, \quad 0 \le f_i \le F_{\max,i}$$

and compares $f_{\text{classical}}$ against the f_{policy} MuJoCo actually produced.

WHY IT IS THIS WAY. Where does τ come from? It is taken as `qfrc_actuator` — the generalised force the muscles actually produced at that instant.
Two reasons. It is *exact*, avoiding the numerical differentiation that inverse dynamics would introduce. And it guarantees a solution exists, because the policy's own force vector already satisfies the constraint — it is a witness. The optimiser can therefore never be asked something impossible.
The consequence is that both methods are posed an *identical* problem on an *identical* trajectory, so any difference is purely load-sharing strategy.

4.6.1 The validity check, and why it is not optional

The whole comparison depends on $R^\top f = \tau$ holding for the policy's own forces. If the moment-arm extraction or the sign convention were wrong, every number would be meaningless while still looking reasonable.

So it is checked and printed every run. Measured: mean residual $1.46 \times 10^{-15}\,\text{N m}$ — machine precision — and 0 optimiser failures in 2000 timesteps.

4.6.2 The metrics it produces

Metric	Meaning
effort ratio	total policy effort ÷ total classical effort. **The headline.** 1.0 = the policy found the minimum-effort solution.
Pearson r	correlation between the two force sets. Mean-centred, so no artificial floor.
pattern cosine	similarity of direction of the 9-vector, ignoring magnitude
null baseline	the same cosine with muscle identities shuffled
per-muscle RMSE	where the disagreement lives, anatomically

WHAT GOES WRONG. Cosine similarity must never be quoted alone. Muscle forces are all non-negative, so two unrelated force vectors already agree substantially by construction. The null — shuffling which muscle each force belongs to, keeping magnitudes — scores 0.373. The observed 0.869 is therefore an excess of +0.496, not a near-perfect 0.87.
Worse, the null's 95th percentile for *individual* timesteps is 0.867, so no single timestep's cosine means anything; only the aggregate does. The null is now computed automatically and written to the output file, so the bare number cannot reappear.

4.7 `run_conventional.py` — the classical baseline

Static optimisation only ever answers the load-sharing sub-question — it is handed a movement. To ask whether classical control could do the *whole job*, a conventional controller was built:

1. Compute a desired hand force: $f_{\text{des}} = K_p(x^\star - x) - K_d\dot{x}$.
2. Map it to joint torques through the Jacobian transpose.
3. **Add gravity, Coriolis and centrifugal compensation.**
4. Probe each muscle's achievable force range at the current state.
5. Solve minimum-effort load sharing over that range.
6. Convert the resulting forces back to activations.

WHAT GOES WRONG. Steps 3 and 4 were both absent initially, and each alone made the baseline a strawman.
Without gravity compensation the arm droops until position error alone generates enough torque to hold it — a large steady-state offset. Measured: it reached only 0.46 m, *worse than a random policy.*
Without the achievable-range probe, the code assumed force is proportional to activation. It is not: force is $\text{gain}(l, v) \cdot a + \text{bias}(l, v)$, state-dependent. Converting a desired *force* to an *activation* requires inverting the muscle model — exactly the operation this thesis proposes to replace. The baseline was skipping the step that makes it conventional.
Publishing "DRL beats classical control" on the uncorrected version would have been a strawman comparison. Both corrections improve the *classical* side.

5 Every Parameter, Explained

This chapter is a reference. Each parameter gets: what it controls, the value used, why that value, and what happens when it is wrong — with a measured consequence wherever one exists.

5.1 Task and episode structure

Parameter	Value	Meaning, rationale, failure mode
`EPISODE_SEC`	2.0 s	Duration of one reach. Long enough to travel 0.8 m and settle; short enough that a million steps buys many attempts.
`FRAME_SKIP`	10	Physics steps per decision. Physics runs at 500 Hz, the policy at 50 Hz. **If too small:** a 2-second episode becomes 1000 decisions, so the budget buys few *attempts* and the policy regresses to the workspace centre — measured, settle spread was 39 % of target spread. **Nearly free:** physics is 6.8 µs/step against 14 ms/gradient update.
`MAX_EPISODE_STEPS`	100	Derived: $2.0/(0.002 \times 10)$. Not set independently.
`EVAL_EPISODES`	50	Episodes per reported evaluation. **Too few and** target-sampling noise dominates — at 50 the success rate still swings 0–20 % across draws.
`EVAL_FREQ`	100k	Learning-curve spacing (`config.py`, used directly by `train.py`). Monitoring only, at five episodes per point — far too noisy to report, and at this spacing a 100k-step run yields a single curve point.

5.2 Reward weights

Weight	Value	Controls / consequence
w_1	1.0	**Distance cost**, quadratic plus linear. The linear part is essential: quadratic alone gives $e^2 = 0.01$ at 10 cm, less than the cost of holding against gravity, so the policy has no reason to close the last stretch. **Observed without it:** hovering at 10–15 cm.
w_2	$1.0 \rightarrow$ swept	**Effort.** Mean of $(f_i/F_{\max,i})^2$. This is the classical criterion in form. **Measured at $w_2 = 1$:** the term contributes 0.051 per step against a precision bonus of 1.441 — roughly **28 times too small to influence anything**. Sweeping it is §6.7.
w_3	0.5	**Smoothness**, mean squared change in command. Mean rather than sum, so it is bounded by 1 like the effort term and the two weights are comparable.

Weight	Value	Controls / consequence
w_4	4.5	**Survival offset.** Paid every step. Chosen so the per-step reward is strictly positive at any reachable error: costs are bounded by $2.7 + 1.0 + 0.5 = 4.2$. **If zero:** quitting becomes optimal — see §6.2.
w_5	1.0	**Success bonus**, paid *per step* on target, not once. Converts the task from reach to reach-and-hold. **If terminal instead:** an alive bonus makes hovering beat succeeding.
w_6	10.0	**Blow-up penalty.** Belt-and-braces on top of forfeited survival reward.
w_7	3.0	**Precision bonus**, $e^{-\|e\|/s}$.
s	0.15 m	**Precision length scale. Originally 0.05, which was a design error:** at the 0.23 m the policy actually occupied, the bonus was worth 0.03 of a reward of 4 — a prize behind a wall. At 0.15 it is worth 0.65 at 23 cm and 2.6 at 2 cm: a smooth pull across the region the policy inhabits.

5.3 Learning hyperparameters

Parameter	Default / tuned	Meaning and consequence
`gamma` (γ)	0.99 / **0.957**	**Discount factor.** Sets the planning horizon, about $1/(1-\gamma)$ steps: 100 at the default, 23 when tuned. **This single parameter accounts for the entire tuning improvement** — ablation recovers +106 % of the gap from it alone, while four others recover nothing. §6.3.
`learning_rate`	3×10^{-4} / 4.4×10^{-4}	Step size for gradient updates. Too high oscillates, too low crawls. **Measured contribution here: −3 %, i.e. none.**
`tau` (τ)	0.005 / 0.0188	**Target-network update rate.** How fast the slow reference copy of the critic tracks the live one. Moved 3.8× in tuning and was the leading suspect. **Measured contribution: −5 %, i.e. none.**
`batch_size`	512 / 128	Samples per gradient update. Smaller is noisier but faster: measured 9.68 ms at 128 against 14.39 ms at 512. **Measured contribution to quality: −5 %.**
`target_entropy`	−9 (auto) / −19.6	**SAC only.** How random the policy is kept. Default is minus the action count. **Believed for weeks to be the cause of the precision plateau; ablation proved it contributes −8 %, i.e. nothing.** §6.3.1.
`buffer_size`	300,000	Replay memory. At 38-dim observations this is ≈106 MB.
`learning_starts`	4,000	Steps of random acting before learning begins, to seed the buffer.
`gradient_steps`	−1 / 8	**The replay ratio control, and a trap.** §5.3.1.

Parameter	Default / tuned	Meaning and consequence
`net_arch`	$[256, 256]$	Two hidden layers. **Deliberately never searched**, because the reported policy footprint (393,750 parameters, latency, memory) is measured for this architecture; searching it would silently invalidate those figures.
`N_ENVS`	4	Parallel simulations. Interacts dangerously with `gradient_steps`.

5.3.1 The replay-ratio trap in detail

WHAT GOES WRONG. With `N_ENVS = 4` and `gradient_steps = 1`, one rollout iteration collects *four* transitions and is followed by *one* gradient update. The replay ratio is therefore 1/4, not the standard ≈ 1.
Measured directly rather than reasoned about:

Configuration	Updates per env step
as originally configured (4 envs, 1 step)	**0.233**
single environment, 1 step (standard)	0.933
4 envs, `gradient_steps` $= -1$	0.933

Every run had therefore performed about 233,000 gradient updates for a nominal budget of 1,000,000 steps — roughly a quarter of the intended training, with no visible symptom. An independent check closes the arithmetic: 233,000 updates at the measured 17 ms predicts 71 minutes, against observed run times of 78.8 and 80.6 minutes.
The fix is `gradient_steps = -1`, which means "as many updates as transitions collected" and therefore holds the ratio at ≈ 1 for *any* `N_ENVS` — the bug cannot silently return if the environment count changes.
The consequence nobody had priced in: correcting it makes training four times more expensive. At a correct ratio, one million steps costs 3.7 hours per seed, so the originally planned 4 algorithms $\times$ 10 seeds would take **148 hours**, not the ≈ 50 assumed throughout. That protocol was never affordable; it only looked so because the runs were under-training.

5.4 Domain randomisation

Each episode, physical properties are perturbed so the controller cannot rely on exact values:

Property	Range	Note
body mass	$\times[0.85, 1.15]$	per body, independently
muscle strength	$\times[0.80, 1.20]$	per muscle, independently
joint damping	$\times[0.70, 1.30]$	per joint
command noise	± 0.02	added to each activation

WHAT GOES WRONG. Two bugs lived here, both silent.
Compounding. The code multiplied the *current* value rather than rescaling from nominal, making it a multiplicative random walk. After about a thousand episodes the arm weighed roughly 1 % of its correct mass. A drift measure where 1.0 means no drift read **0.013**; after the fix, **0.998**.
One number for everything. The random draw was made without a `size` argument, returning a single scalar applied to every body and every muscle. Instead of independent per-element variation there was one global scale factor — which a policy can trivially detect and cancel, making the robustness claim vacuous.

5.5 Evaluation parameters, and a subtlety that mattered

WHAT GOES WRONG. Four defects in the evaluation path, each verified empirically:
1. The evaluation seed was dead code. `evaluate(seed=...)` never referenced its argument; `load_model` fixed the environment seed at zero. Two loads with different seeds returned byte-identical results.
2. `reset(seed=...)` does not reseed target sampling. Targets come from a generator seeded only at construction; the standard Gymnasium reset seeds a *different* generator this environment never uses. Targets can only be changed at construction.
3. Two resets per episode. The loop reset the vectorised environment at the top of each iteration while the wrapper also auto-resets on termination. Instrumented and confirmed: evaluation scored on every *other* target, a different test set from the force and conventional analyses — which made those comparisons unpaired.
4. A diverged episode could be scored a success, because joint speed is forced to zero on divergence, automatically satisfying the "settled" half of the criterion.
The consequence. Every interval reported was *training-seed spread on one fixed draw of 50 targets*. Measured across five target draws for one policy: final error 0.1130±0.0146 against 0.1033± 0.0034 across training seeds — **the target draw contributes about four times more variance**. And `reach_5cm` ranged 0.16–0.34 across draws, with the reported 0.34 being the maximum.
What it does not undermine. Because the seed was fixed, every configuration was evaluated on *identical* targets, so all between-configuration comparisons are *paired* — the stronger design, arrived at by accident. The comparisons stand; the absolute values needed wider intervals, and everything has been re-measured across three target seeds.

5.6 Hardware, and why it constrained the science

CPU	AMD Ryzen 7 4800H, 8 physical cores / 16 logical
RAM	15.4 GB
GPU	GTX 1660 Ti — present, and *slower* than the CPU here
Thread budget	capped at 8 of 16
Total compute	≈ 60 hours

WHAT GOES WRONG. The machine **stopped responding entirely** under four concurrent training workers: no crash dump despite dumps being enabled, no hardware error logged, and the kernel unable to write its own event log. The evidence points to a thermal or power limit rather than a software fault — single-worker runs of 78–80 minutes had completed without incident. It cost a full day.
All subsequent work is capped at 8 of 16 logical processors. Benchmarking then showed the parallelism had bought almost nothing anyway: 8 threads are only 1.78× faster than 1, so the four workers were starving each other while saturating the processor.

WHY IT IS THIS WAY. **Why the GPU does not help.** Measured on the real training path: CPU 9.68 ms per update, GPU 15.00 ms — the GPU is 1.5× *slower*.
The reason is measurable too. A trivial operation costs 7 µs on the CPU and 26.4 µs on the GPU; a full three-layer forward pass costs 167 µs on the GPU, of which roughly 156 is six kernel launches. **The arithmetic is essentially free; you pay almost entirely for the privilege of asking.** With networks this small and hundreds of tiny operations per update, that launch overhead dominates.
Running CPU and GPU *concurrently on different jobs* is a different question and does help: measured 1.43× total throughput, because the jobs are independent and the GPU worker needs only about 0.9 of a CPU core.

6 The Results, and Why They Came Out That Way

This chapter takes each result in turn and answers three questions about it: **what** was measured, **why** it turned out that way, and **how much confidence** it deserves. Where a result replaced an earlier one, the earlier one is shown too — the direction in which a number moves when it is checked is itself information.

6.1 How performance is measured, and why not by success rate

Metric	Definition and what it tells you
Final error	Hand-to-target distance at the last step. **The headline number:** it measures reaching *and* holding.
Closest approach	Smallest distance at any instant. Separates "can it get there" from "can it stay".
Ends within 5 / 10 cm	Graded proximity. Stable where a strict threshold is not.
Drift	Closest approach minus final error — how much is given back after arriving.
Path length	Distance travelled per 0.80 m of required travel. Exposes sweeping and overshoot.
Energy	Summed squared activation. Distinguishes a passive policy from a straining one.
Success rate	Within 2 cm *and* settled. **Reported, never headlined.**

WHY IT IS THIS WAY. Why success rate is not the headline. Across five runs of one identical configuration it read 0 %, 4 %, 6 %, 8 % and 20 %. A measure that varies twentyfold under identical settings is reporting sampling noise, not performance.
The reason is structural: 4 % of fifty episodes is *two episodes*. Any binary threshold near the edge of a system's capability behaves this way. Final error averages over all fifty and is stable to 4 % across the same five runs. This is why the graded proximity measures exist — they retain the interpretability of a threshold without its brittleness.

6.2 The self-destruction exploit

What was measured. Fifty evaluation episodes of a policy that appeared, from its reward curve, to be learning. **Result: 100 % ended in a deliberate numerical blow-up at step 7 of 100.** Not one episode ran to completion.

WHY IT IS THIS WAY. Why it happened. Every per-step reward term was zero or negative, and a blow-up terminated the episode with no penalty attached. In this framework a terminated episode accrues nothing further — its remaining value is exactly zero — while surviving all 100 steps accumulated about −20.
Quitting was worth 20 more than participating. The agent found the cheapest way to quit: drive the muscles into a configuration the integrator cannot handle.
The algorithm was not broken. It maximised the objective it was given, precisely. The objective was wrong.

The fix, and why it is the shape it is. A survival term $w_4 = 4.5$ paid *every step*, chosen so the per-step reward is strictly positive at any reachable error (costs are bounded by $2.7 + 1.0 + 0.5 = 4.2$), plus an explicit blow-up penalty $w_6 = 10$. Living now strictly beats quitting from every state — not on average, but pointwise, which is the property that makes the fix robust rather than lucky.

Confidence: certain. The failure was directly observed on 50 of 50 episodes and the blow-up rate is exactly zero across all five replication seeds afterwards.

6.3 The discount factor is the whole story

This is the most important result in the project, and it was arrived at by first being wrong.

6.3.1 The wrong answer, held for weeks

A sixteen-trial Bayesian search over five hyperparameters produced a configuration that cut final error from 0.261 to 0.161 m. All five best trials selected a target entropy near -19.6 against the library default of -9.

WHY IT IS THIS WAY. **Why that looked conclusive.** The mechanism fit the symptom exactly. SAC pays its policy to stay random; residual randomness is precisely what stops an end effector settling; the default scales with the *number of muscles* rather than with anything about the task; and every good trial independently agreed on roughly half the default randomness.
Four independent lines of evidence, all pointing one way. It was written up as a contribution of the thesis.

WHAT GOES WRONG. **It was wrong.** A two-sided one-at-a-time ablation, three seeds per arm:

Configuration	Final error (m)	Gap closed
All defaults *(reference)*	0.2613 ± 0.0203	0 %
Defaults + tuned entropy only	0.2688 ± 0.0090	−8 %
Tuned − entropy reverted	0.1552 ± 0.0181	+106 %
All tuned *(reference)*	0.1610 ± 0.0084	100 %

Adding the tuned entropy to defaults recovers *nothing*. Removing it from the tuned configuration costs *nothing*. Both directions agree, so this is not a marginal call.
Why the evidence misled. A TPE sampler concentrates its proposals where performance is good. Parameters therefore cluster in the best trials whether or not they are causally responsible. Reading causation from that clustering is a plain error — and it is the error this project made.
A hyperparameter search identifies a configuration, never a cause. Only a deliberate ablation separates the two.

6.3.2 The right answer

The same procedure applied to all five parameters:

Defaults, plus one tuned parameter	**Final error (m)**	**Gap closed**
(nothing changed)	0.2613 ± 0.0203	0 %
target entropy only	0.2688 ± 0.0090	−8 %
learning rate only	0.2642 ± 0.0026	−3 %
τ only	0.2660 ± 0.0117	−5 %
batch size only	0.2658 ± 0.0096	−5 %
discount factor only	$\mathbf{0.1553 \pm 0.0061}$	**+106** %
All tuned *(reference)*	0.1610 ± 0.0084	100 %

Five parameters tested individually. Four do nothing. One explains everything.

IN PLAIN LANGUAGE. The discount factor decides how far into the future the agent bothers to look — about $1/(1-\gamma)$ steps ahead.

	γ	**effective horizon**
Library default	0.99	100 steps — the whole episode
Selected by the search	0.957	23 steps = 0.46 s

Now compare that with what the arm actually does. Measured from the trajectory, the hand arrives at the target by **step 25** — 0.50 s.
The tuned value matches the duration of the movement almost exactly. The default was four times too long.

WHY IT IS THIS WAY. Why a too-long horizon hurts, mechanically. At $\gamma = 0.99$ the value of an early step includes seventy-odd later steps of holding reward. That is a high-variance quantity to estimate. A critic fed high-variance targets produces unreliable estimates; an actor trained against an unreliable critic cannot refine its endpoint. The visible symptom is an evaluation curve that oscillates and never sharpens — exactly what every untuned run in this project showed.
Shortening the horizon to the length of the reach makes credit assignment local and the value estimate tractable.
The transferable statement, which is the part worth carrying to another project: *match the discount factor to the timescale of the behaviour, not to a library default. Where the default horizon greatly exceeds the task duration, the resulting variance presents as an inability to refine — a symptom easily and wrongly blamed on insufficient training, on reward shaping, or on exploration.* This project spent considerable effort on all three before testing γ.

Confidence: high. Single parameter changed, three seeds, full effect recovered in both directions, four alternatives excluded by the identical procedure, and a mechanism that predicts the observed timescale quantitatively.

6.4 What the tuned policy actually does

Metric (300k steps, 50 episodes)	Default	Tuned
Final error (m)	0.204	**0.106**
Closest approach (m)	0.132	**0.074**
Drift (m)	0.072	**0.035**
Ends within 10 cm	18 %	**58 %**
Energy	774 667	**276 804**
Path length per 0.80 m reach	3.55 m	1.64 m

IN PLAIN LANGUAGE. Read the path-length row. To cover 0.80 m of required travel, the untuned policy moved its hand 3.55 m — it swept around, overshot, and came back. The tuned policy moved 1.64 m, with net displacement 0.99× the distance required.
It stopped wandering and started going there.

Intra-episode, averaged over thirty episodes: the hand starts 0.795 m away, is at 0.198 by step 10, 0.119 by step 25, and then holds — 0.110, 0.112, 0.113 m at steps 50, 75, 99. **That flat tail is the whole point:** converged and stationary, not still drifting.

6.5 Comparing the four algorithms, and getting it wrong first

What was measured first. Five configurations, three seeds, 100k steps. Ordering: SAC < TD3 < DDPG < PPO. DDPG was reported as clearly the weakest off-policy method, explained by its lack of twin critics — the textbook account.

WHAT GOES WRONG. That comparison was invalid, and by its own later findings. SAC ran with $\gamma = 0.957$; the other three ran at the library default 0.99. Once γ was shown to be the decisive parameter, the benchmark stood revealed as *one correctly configured entrant against three handicapped ones.*
Retrained with $\gamma = 0.957$ for all four:

	Before (γ=0.99)	After (γ=0.957)	Change
SAC	0.2613 ± 0.0203	$\mathbf{0.1909 \pm 0.0341}$	−27 %
DDPG	0.4161 ± 0.0189	$\mathbf{0.2510 \pm 0.0383}$	**−40 %**
TD3	0.2786 ± 0.0110	0.2579 ± 0.0348	−7 %
PPO	0.4231 ± 0.0351	0.4661 ± 0.0401	+10 %

DDPG improves 40 % and overtakes TD3. The ranking changes. **The twin-critic explanation was constructed after the fact around a result that has not survived** — DDPG was not failing for want of twin critics; it was failing because its horizon was wrong, and it suffered more from that than the others. Corrected, it is indistinguishable from TD3 and no ordering between them is supported.

WHY IT IS THIS WAY. Why PPO got worse, uniquely. PPO estimates advantages through generalised advantage estimation, where γ and λ *jointly* set the bias–variance trade-off. Lowering γ while leaving $\lambda = 0.95$ shortens the horizon on one side of a pair only. The correct adjustment for PPO touches both, and was not attempted — so PPO's last place is again about configuration, not about the algorithm.
This is what makes fair comparison of RL algorithms genuinely hard: there is no single setting that can be held equal across all four, because the parameters do not mean the same thing in each.

The conclusion that survives. The gap between the best and second-best off-policy method is 0.060 m. The improvement obtained in DDPG by correcting γ alone was 0.165 m — roughly **three times the gap between algorithms**.

On this task, choosing the discount factor correctly mattered about three times more than choosing between SAC, TD3 and DDPG.

Confidence: moderate. Three seeds, no significance testing (three seeds cannot support a signed-rank test with useful power, so none is claimed), a 100k-step early-training snapshot, and only γ corrected — a *fair* comparison, not a *tuned* one. A tuned comparison needs a separate search per algorithm, which the compute budget did not allow.

6.6 The central result: do the muscle forces agree?

This is the thesis question. Everything above concerns whether the arm reaches; this concerns whether it reaches *the same way*.

6.6.1 How the comparison is made fair

At every timestep of a rollout, take the joint torque τ the policy actually produced, hand it to static optimisation, and ask what forces *it* would have chosen for that same torque.

WHY IT IS THIS WAY. Why τ is taken from the policy rather than computed independently. Two reasons, and both are about eliminating excuses.
It is *exact* — `qfrc_actuator` is the generalised force the muscles genuinely produced, so no numerical differentiation enters. And it is *guaranteed feasible* — the policy's own force vector satisfies the constraint by construction, so static optimisation can never fail for want of a solution.
Both methods are then posed an identical problem on an identical trajectory. Any difference is purely a difference of load-sharing strategy, which is exactly what the thesis asks about.
The cost of this choice, stated plainly: the classical method never chooses a trajectory of its own. So this validates load sharing *along a given path*, not the path. A controller could move in a way no human would and still share load classically. Trajectory realism is not assessed anywhere in this work.

The identity was verified, not assumed. Over 2000 timesteps, the mean residual $\|R^\top f - \tau\|$ is 1.46×10^{-15} N m for the policy's forces and 1.68×10^{-15} for the classical solution, with 0 optimisation failures. Both at machine precision. A sign error or a moment-arm extraction error would have invalidated the whole chapter, and would have shown up here.

6.6.2 Result one: the pattern agrees

Pearson correlation, all muscles	$\mathbf{0.81 \pm 0.04}$
Pattern cosine similarity	0.813 ± 0.047
null baseline	0.372 ± 0.017
excess over null	$\mathbf{+0.441}$

WHAT GOES WRONG. The cosine number cannot be quoted on its own, and this nearly went wrong. Muscle forces are non-negative by construction, so two *completely unrelated* force vectors already agree strongly — all their components point into the same corner of the space. The correct null: at each timestep, permute which muscle each of the policy's own force magnitudes belongs to. This destroys the correspondence while preserving that instant's magnitude distribution exactly.

That null averages **0.373**. So of an apparent 0.813, roughly the first 0.37 is a floor imposed by arithmetic, not evidence of anything.

A second-order error inside the first. The null was initially computed on *time-averaged* force vectors and gave 0.767 — which would have made the result look like nothing at all. That was also wrong: averaging over time removes the very variation the statistic is meant to test. Computed per timestep, as it must be, it is 0.373.

And because the null's 95th percentile for *individual* timesteps is 0.867, no single timestep's cosine means anything. Only the aggregate does.

The Pearson correlation is mean-centred, carries no such floor, and is the figure to quote when one number is required.

6.6.3 Result two: the economy does not agree

Producing identical joint torques, the learned controller expends $\mathbf{1.91 \pm 0.15}$ times the effort of the minimum-effort solution.

Where the excess sits — it is not spread evenly:

Muscle	Policy (N)	Classical (N)	r	NRMSE
deltoid anterior	63.7	59.2	0.959	2.5 %
deltoid medial	145.3	112.3	0.954	6.1 %
deltoid posterior	35.5	38.7	0.859	10.0 %
triceps long	180.4	134.0	0.910	11.2 %
biceps long	52.1	41.6	0.893	7.5 %
brachialis	113.9	87.6	0.871	8.6 %
biceps short	90.7	45.7	0.764	18.1 %
triceps lateral	79.1	25.8	**0.416**	16.5 %
triceps medial	64.3	14.2	**0.416**	15.3 %

IN PLAIN LANGUAGE. The three shoulder muscles agree closely — one to within 2.5 %. The problem is in two elbow extensors, where the policy uses three to four and a half times what is prescribed.

Those muscles *straighten* the elbow while the biceps group *bends* it. Driving both at once produces largely cancelling torques. The elbow ends up at roughly the right angle, but both groups have spent large forces to achieve what one alone would have done.

This is **co-contraction**. Humans do it deliberately when a joint needs to be stiff — carrying a full cup, or steadying a hand. Doing it throughout an unloaded reach is simply waste, and it is not what the minimum-effort criterion selects.

Independently corroborated. The Falconer–Winter co-contraction index over the elbow flexor/extensor groups reads 0.564 — elevated, and the only behavioural metric that did *not* improve under tuning (0.501 before). Two methodologically unrelated analyses, one a comparison against classical optimisation and one a standard EMG-derived index, localise the same defect to the same muscle group. That agreement is stronger evidence than either alone.

6.6.4 Result three: the task is solved consistently, the muscle solution is not

Five identical runs differing only in random seed:

Seed	Final error	Closest	Effort ratio	r	Success
0	0.1142	0.0766	1.71	0.865	0.04
1	0.1086	0.0785	1.91	0.823	0.20
2	0.1034	0.0757	2.12	0.759	0.00
3	0.1066	0.0672	1.86	0.825	0.08
4	0.1041	0.0691	1.97	0.767	0.06
mean	0.1074	0.0734	1.914	0.808	0.076
std	0.0043	0.0050	0.150	0.044	0.075

Two facts sit side by side in that table and their combination is the most informative result in the thesis. **Final error varies by 4 % across the five runs. Muscular energy varies by 77 %** (273,241 to 483,373).

IN PLAIN LANGUAGE. Five controllers, trained identically apart from a random seed, all arrive at essentially the same place — using materially different sets of muscle forces.
That is the redundancy problem of Chapter 1 appearing directly in the results. Nine muscles, four joints; many force distributions give the same movement; and nothing in the reward chose among them. The learner settled somewhere different each time.
It also tells you where the fix is. Huge variance in the muscular solution alongside near-constancy in the task outcome is the signature of **an objective that is undetermined in exactly the dimension you care about.**

WHAT GOES WRONG. The originally reported run was the best of the five. Seed 0 — the run every earlier section analysed — gave the *lowest* effort ratio (1.71 vs a mean of 1.914) and the *highest* correlation (0.865 vs 0.808). On both metrics carrying the argument, the single run first reported was the most favourable of five.
Not a large effect, but a systematic one, and exactly the error that reporting a single run invites. Without replication this thesis would have reported both an agreement and an efficiency better than the method delivers.

6.7 Closing the gap: the effort weight

The prediction, made before the experiment. If the effort term is 28 times smaller than the precision bonus, it cannot be influencing the solution — so raising w_2 should reduce the discrepancy, at some cost in accuracy.

Run at $w_2 \in \{1, 5, 20\}$, everything else fixed, five seeds each, full 300k-step budget, and evaluated under the *default* weights so the numbers stay comparable with every other result:

Config.	Effort ratio	Pearson r	Final error	Energy
$w_2 = 1$	1.914 ± 0.150	0.808 ± 0.044	0.1074 ± 0.0079	327 048
$w_2 = 5$	1.406 ± 0.092	0.919 ± 0.016	0.1114 ± 0.0053	104 598
$\mathbf{w_2 = 20}$	$\mathbf{1.304 \pm 0.025}$	$\mathbf{0.943 \pm 0.008}$	0.1161 ± 0.0108	**37 076**
Conventional	1.263	0.931	0.165	102 876

The prediction held in the means. The per-seed ranges tell a more careful story, and it is worth getting right.

	the five seeds	**range**
$w_2 = 1$	1.714, 1.856, 1.912, 1.974, 2.116	1.714–2.116
$w_2 = 5$	**1.336**, 1.351, 1.355, 1.429, 1.557	1.336–1.557
$w_2 = 20$	1.278, 1.292, 1.293, 1.318, **1.345**	1.278–**1.345**

WHAT GOES WRONG. An earlier version of this document said the three distributions were *cleanly separated*. Checked against the per-seed numbers, that is true of one step and false of the other.
$w_2 = 1 \to 5$ **is** clean: every seed of one lies below every seed of the other, $1.557 < 1.714$.
$w_2 = 5 \to 20$ **is not**: the worst $w_2 = 20$ seed at **1.345** sits just above the best $w_2 = 5$ seed at **1.336**. The ranges overlap by 0.009.
The means still move the right way, and four of the five $w_2 = 20$ seeds do clear the entire $w_2 = 5$ range — but one overlapping pair at $n = 5$ makes that second step a *probable* further improvement, not a demonstrated one. The upper bound had also been mis-transcribed as 1.340.

Per-muscle RMSE falls from 10.1 % to about 3 % of mean $F_{\max}$.

The accuracy cost is real but modest: 8 % worse final error for a 32 % reduction in wasted effort. Closest approach is unchanged (0.0734, 0.0750, 0.0737), so what is lost is *holding*, not *reaching*.

WHY IT IS THIS WAY. The most informative row is the standard deviation. It collapses: $0.150 \to 0.092 \to 0.025$ on effort, and $0.044 \to 0.016 \to 0.008$ on correlation.
That confirms the interpretation offered above. At $w_2 = 1$ the muscular solution varied 77 % across seeds while task performance stayed constant, because the objective did not constrain it. Weighting that dimension *pins the solution down*: at $w_2 = 20$ the load-sharing outcome is determined by the objective rather than by the random seed.
This is a stronger form of evidence than the mean improvement alone. It shows the mechanism, not just the effect.

The headline answer changes. It is not that DRL reproduces coordination but not economy. It is that DRL reproduces **both** — provided the objective asks for both. The version originally reported did not ask.

6.7.1 Two predictions, one right and one wrong

Both are recorded because the direction of the error is instructive.

Right: the prediction that raising w_2 would trade accuracy for fidelity, made in advance, borne out in both directions.

Wrong: this author predicted that $w_2 = 20$ would *regress* on replication, reasoning that single-seed results in this project had twice proved optimistic. It did the opposite — seed 0's 1.34 was the *worst* of the five and the mean is 1.304. Selection bias explains a favourable first measurement; it does not license assuming that every first measurement is favourable.

6.8 Against a conventional controller — and a withdrawn claim

Static optimisation is an *idealised* reference: solved instant by instant, with no actuator lag and no tracking. Comparing against it says how far the policy is from a mathematical optimum, not

how far it is from what conventional control actually achieves — and the thesis question is the latter.

A Cartesian impedance controller was therefore built: $f_{\text{des}} = K_p(x^\star - x) - K_d\dot{x}$, mapped to joint torques by the Jacobian transpose, with gravity and Coriolis terms compensated, and distributed onto muscles by minimum-effort load sharing over the achievable force range. Both gains were grid-searched, because a tuned policy against an untuned controller is not a test.

WHAT GOES WRONG. **Two defects would have made it a strawman.** Both are on the *classical* side, and both were found and fixed before publishing anything.
No gravity compensation. Without the bias term the arm droops until position error alone generates enough torque, leaving a large steady-state offset. Measured, it reached only 0.46 m — *worse than a random policy*. Any textbook impedance controller includes this term.
It assumed force is proportional to activation. It is not: force is $g(l, v)\, a + b(l, v)$, state-dependent. Converting a desired force to an activation requires inverting the muscle model at the current length and velocity. Probing at $a = 0$ and $a = 1$ recovers that affine relation exactly.
Reporting the comparison without these fixes would have overstated the case for learning. This is the single easiest way to fake a machine-learning result, and it is usually done by accident.

6.8.1 Why the conventional controller is not 1.00

It solves minimum-effort load sharing explicitly at every timestep, so its effort ratio "should" be exactly 1.00. It is **1.263**.

WHY IT IS THIS WAY. The gap is not an error — it is the price of closed-loop control. Activation dynamics mean the commanded activation is not the realised one; the controller acts at 50 Hz rather than continuously; and it tracks a moving state rather than solving a static instant.
The idealised 1.00 is not attainable by any controller operating under these constraints, and this reframes the central number. Measured against 1.00, the learned policy appears to waste 91 % of its effort. Measured against 1.263 — the achievable floor, demonstrated by a controller that optimises effort explicitly — it uses about **51 % more than necessary**. The second figure is the meaningful one.

6.8.2 A claim made, then withdrawn

WHAT GOES WRONG. An earlier draft observed that the learned policy at $w_2 = 20$ reached 1.304 ± 0.025, *below* the 1.33 then measured for the conventional controller — and cautioned that the 1.33 rested on a single gain pair from a coarse twelve-point grid, so a better-tuned controller might overturn it.
A thirty-point search was run. It did. The coarse search had bottomed out against its own lower boundary; the better gains are *gentler* in both stiffness and damping ($k_p = 20$, $k_d = 6$ against $k_p = 60$, $k_d = 25$), and give both better accuracy (0.165 vs 0.199 m) and better economy (1.263 vs 1.330).
The claim that a learned policy distributes force more economically than a controller optimising that quantity in closed form is therefore not supported, and the paragraph advancing it was removed.

The honest comparison, both sides measured properly:

	Learned ($w_2 = 20$)	**Conventional**
Final error	$\mathbf{0.1072 \pm 0.0148}$	0.165 m
Effort ratio	1.304 ± 0.025	**1.263**
Agreement with classical (r)	$\mathbf{0.943 \pm 0.008}$	0.931

Neither method dominates. The learned policy reaches about 35 % more accurately; the conventional controller is about 3 % more economical — a margin comparable to the learned policy's own seed spread, so the two are close, but the ordering on effort favours the classical method.

What survives is narrower and sturdier than what was first written: *at an appropriate effort weight, a learned policy achieves load-sharing fidelity comparable to a properly tuned conventional controller, while reaching substantially more accurately.*

6.9 The speed argument, withdrawn

The original motivation was inference cost. First measurement: classical 2165 µs against the network's 295 µs, a 7.3× advantage.

WHAT GOES WRONG. Two objections, and together they are fatal.
It timed the wrong computation. The 2165 µs is the Newton–Raphson solve of Hill fibre–tendon equilibrium, which is part of *forward simulation*. The question this thesis asks is answered by *load-sharing optimisation* — a different calculation entirely. The two had been conflated.
Both sides were unoptimised Python. Re-timed like-for-like, with moment-arm extraction charged to the classical side:

	mean (µs)	**p99**
Network forward pass	288.8	499.9
Classical, general solver (SLSQP)	2370.5	5133.1
Classical, direct solve + moment arms	**23.9**	—

Against a general-purpose solver the network is 8.2× faster. Against a direct solve of the actual problem it is **about twelve times slower**.
The latency argument is withdrawn. The 7.3× reflected the cost of one Python implementation, not a property of the two approaches.

What survives, honestly stated. A qualification runs the other way: the box constraints are active in 100 % of sampled states, so the closed-form solve is not exactly valid and a constrained active-set solver would cost more than 23.9 µs. It would not cost 2370. The defensible statement is that the two are *comparable* in cost.

What genuinely survives is the *shape* of the cost, not its magnitude: the network's runtime is fixed and branch-free regardless of biomechanical state, where the iterative solver's depends on convergence. Where a hard real-time bound matters more than mean latency, that retains value. It is a considerably weaker claim than a speed-up, and it is the one made.

The policy contains 393,750 parameters — 1.5 MB single-precision, 385 kB quantised to eight bits. These figures are independent of policy quality, and specific to the SAC architecture (PPO's is 154,131).

6.10 Muscle synergies, and the weakest of the three analyses

Motor-control theory holds that the nervous system manages redundant musculature through a few co-activated groups rather than muscle by muscle. Non-negative matrix factorisation recovers that structure; VAF measures how much it explains.

Two findings and one caution.

The classical solution is uniformly more low-dimensional. Static optimisation needs four or five synergies to reach 90 % VAF where the learned policies need six to nine. This is

co-contraction expressed in synergy space: driving an antagonist independently of its agonist requires an extra dimension that the classical solution never uses.

Agreement between learned and classical synergy structure tracks training quality — 0.676 for default SAC against 0.851 for tuned SAC at 300k.

WHAT GOES WRONG. But this statistic is unstable, and the first version of the result exploited that without meaning to. Varying the episode count for one fixed policy gave 0.873, 0.805, 0.950, 0.810, 0.739 at 8, 12, 15, 20, 25 episodes — a spread of 0.21, and non-monotone, so it is sampling noise, not convergence.
The VAF over the same range moved only 0.818–0.870, which localises the problem: the instability is in the *basis matching*, not the decomposition. NMF has no unique solution, and two independently fitted bases inherit that ambiguity.
Re-estimated over six random 15-episode subsets, the reported 0.950 becomes 0.851 ± 0.129 — and 0.950 sits near the top of its own resampled range $[0.708, 0.979]$.
A separate methodological error in the same statistic. The similarity was originally a best match per synergy, which lets several extracted synergies claim the same reference while others go unmatched. On real data only three of four references were distinctly matched, inflating the score by roughly 0.10. All figures now use a one-to-one (Hungarian) assignment, which is what a correspondence between sets actually means.

Confidence: low, and labelled as such. The ordering survives — the gap of 0.175 exceeds the pooled spread — so the qualitative claim stands. But this is the weakest of the three convergent analyses and should be weighted accordingly: the effort ratio (five seeds, non-overlapping distributions) and the co-contraction index are far better determined than a statistic whose sampling spread is comparable to the effect it measures.

The comparison against *human* synergies (0.62–0.80) is not interpreted at all: the reference loadings are a documented qualitative approximation over this model's muscle ordering, not digitised from the original publication.

6.11 Confidence: what is solid, and what is not

Claim	Confidence	Basis
γ causes the improvement	**high**	two-sided ablation, 4 alternatives excluded, mechanism predicts timescale
Replay ratio was 0.233	**high**	directly counted; run-time arithmetic closes
Load-sharing pattern agrees	**high**	5 seeds, proper null, machine-precision validity check
Excess effort is co-contraction at the elbow	**high**	two independent analyses localise it identically
Raising w_2 closes the gap	**high**	predicted in advance, 5 seeds, non-overlapping distributions, variance collapses
Learned vs conventional trade-off	**moderate**	both sides tuned, but $n = 1$ controller at one gain pair
Algorithm ordering	**moderate**	3 seeds, no significance test, 100k snapshot, untuned
Synergy agreement tracks quality	**low**	sampling spread comparable to the effect
Anything about *human* motor control	**none**	no EMG, no subject, no measured force

6.11.1 The five limitations that matter most

1. No ground truth exists anywhere in this work. Both sides are computed in the same simulator on the same muscle model. Where they agree, two computations over one approximation agree. And static optimisation is itself a *theory* of load sharing — well established, still contested. A controller that disagrees with it is not thereby shown to be unphysiological.

2. One task. A single reaching movement in one workspace. Nothing here establishes that any finding transfers to another task — including the γ result, whose mechanism explicitly depends on this task's duration.

3. Three seeds, no significance testing. Where five seeds exist they are used; where three do, no test is claimed, because three cannot support one.

4. "Complete Hill model" overstates what is active. The parallel elastic element returns exactly zero in every sampled state, and the force–length curve is exercised only on its ascending limb.

5. Trajectory realism is not assessed. The comparison validates load sharing along a given path, never the path itself.

6.12 Questions people actually ask

Did the DRL work? On reaching: yes — a hand starting 0.80 m away ends within 0.107 m and holds. On the thesis question: it reproduces *which* muscles act and in *what proportion* ($r = 0.94$ at $w_2 = 20$), and it reproduces the *magnitude* too once the objective asks for it.

Is 0.107 m good? It is not human-level, and this document does not claim it is. For context: the untuned baseline was 0.26 m, a properly tuned conventional controller reaches 0.165 m, and a controller given *full privileged state* has a median final error of 0.08 m. The learned policy, acting only on proprioception, sits between the conventional controller and the privileged one.

Why is the success rate so low? Because it demands 2 cm *and* settled, and 4 % of fifty episodes is two episodes. Across five identical runs it read 0, 4, 6, 8 and 20 %. It is noise, and it is reported as noise.

Should we use DRL instead of classical calculation? Not for this, on this evidence. It is not faster (that claim is withdrawn), and it needs the effort objective supplied explicitly, which is the classical criterion — so it does not remove the modelling assumption, it relocates it. What it offers is a fixed, branch-free inference cost and better endpoint accuracy than a conventional controller. That is a trade-off, not a replacement.

What is the single most useful thing here? That the discount factor, matched to the timescale of the behaviour, mattered about three times more than the choice of algorithm — and that this was invisible until an ablation was run, having been confidently misattributed to something else first.

Why does the document keep describing its own mistakes? Because every one of them changed a number that had already been written down, and in every case the uncorrected number was the more flattering one. A result that has survived a serious attempt to break it is worth more than one that has not been tested; showing the attempts is how a reader can tell which kind they are holding.